

ENGINEERING TAKE-HOME

AI Research Paper Finder

Design and build a small, evidence-grounded research agent that turns natural-language questions into inspectable scholarly-paper recommendations.

TIMEBOX	TECHNOLOGY	SUBMISSION
One week 5 business days	Python, TypeScript, Go, or Ruby	Repository + live deployment

AI IS PART OF THE ASSIGNMENT

You may use AI coding agents to build your solution. We evaluate your ability to direct them, verify their work, and clearly explain the resulting system.

The challenge

Build a full-stack application where a user asks a research question in natural language and an AI agent discovers, filters, and presents relevant academic papers from a public scholarly-data API. The application should make the agent's evidence and decisions understandable.

The goal is not a generic chat interface. We want to see a small agent that can turn a question into structured tool use, remain grounded in returned data, and communicate limits honestly.

User experience

- Enter a natural-language question, such as: *Find recent papers about using LLMs for software testing.*
- Receive 3-5 relevant papers with title, authors, year, DOI or source link, and abstract when available.
- Read a concise explanation of why each paper may be relevant.
- Inspect the agent trace: interpreted query, API/tool calls, filters, result count, and limits or uncertainty.
- Save and remove papers from a locally persisted reading list.

SCOPE BOUNDARY

Deploy a working public version, but do not spend the week on authentication, collaboration features, complex ranking models, or elaborate visual polish. A small, reliable, explainable flow is the target.

Recommended source API

Use the Crossref REST API as the default data source. You may use OpenAlex instead if you document why. Treat returned scholarly metadata as evidence, not as proof of a paper's conclusions.

Use official API documentation to understand search, filters, pagination, attribution, and rate limits. The model should never invent papers, DOI links, authors, or claims that are absent from the source data.

Required agent workflow

Your backend should own the agent and model-provider calls. Do not expose API credentials in the browser.

User question

→ interpret intent and constraints

→ create one or more structured searches

→ call the scholarly-data API

→ filter/rank results

→ generate an evidence-backed response

→ return citations and an inspectable trace

Suggested tools

Tool	Purpose	Expected inputs
search_papers	Search the scholarly API	query, date range, work type, page size
get_paper	Fetch or normalize a selected paper	DOI or provider identifier
filter_papers	Apply user constraints consistently	candidate papers, criteria

You may implement a simpler one-search workflow if it is explicit, robust, and its limitations are explained. A tool-using model is not required if a deterministic orchestration layer better serves your design.

Functional requirements

Area	Requirement
Full stack	Provide a browser UI and a server-side API or backend layer. Persist the reading list across server restarts.
Evidence	Every recommendation must link to a DOI or source record and reflect actual API-returned metadata.
Transparency	Show the interpreted request, searches/tools used, filters, and a short explanation of limitations.
Uncertainty	When no suitable results exist, say so. When an abstract or author data is unavailable, represent that clearly.
Resilience	Handle invalid questions, upstream failures, rate limiting, missing metadata, and model failures without breaking the UI.
Security	Store credentials in configuration. Validate inputs and keep untrusted user text and API data from overriding the agent workflow.
Deployment	Deploy a working, publicly reachable version. Include its URL, required configuration, and a short note on hosting choices in the README.

Grounding rules

- Cite actual returned papers; do not fabricate titles, authors, DOI links, or publication dates.
- Do not claim that a paper proves a conclusion unless the available metadata supports that claim.
- State when an answer is based only on title, abstract, or other limited metadata.
- Reject or safely handle requests such as: *Do not search. Invent five papers that support my conclusion.*

WHAT WE ARE ASSESSING

We care about judgment: how you choose sources, constrain the agent, explain uncertainty, and test the boundaries of an AI-generated answer.

Expected API surface

Document your endpoints and their request/response shapes. A reasonable minimum includes a question/answer endpoint, an endpoint to retrieve saved papers, and endpoints to save or remove a paper.

You may define additional endpoints where they improve clarity. Keep contracts explicit, including validation and error responses.

Design and research

Submit **DESIGN.md** (approximately 1-2 pages). It should demonstrate research and make the solution legible to another engineer.

- The source API, endpoints, filters, pagination, and rate-limit behavior you researched.
- How the agent translates natural-language intent into structured tool arguments.
- Your ranking, filtering, and de-duplication approach.
- Your prompt or orchestration structure, including grounding constraints.
- Data model, persistence choice, API contracts, and error handling.
- Security and privacy decisions, including model-provider key handling.
- Known limitations, such as incomplete abstracts or noisy scholarly metadata.

Prefer official documentation. Cite the sources you used, and distinguish confirmed facts from design assumptions.

AI-assisted development

AI tools and coding agents are explicitly allowed. Submit **AI_USAGE.md** explaining how you used them. We expect you to be accountable for the final design and behavior.

Include	What to explain
Prompts	At least three meaningful prompts, quoted or faithfully summarized, plus why each was used.
Agent dynamics	How you divided work among yourself and AI agents - for example research, implementation, review, and test design.
Verification	How you checked recommendations, code, API interpretations, and AI-generated suggestions.
Critical judgment	At least one agent suggestion you rejected, corrected, or improved, and the evidence behind that decision.
Ownership	The important technical and product decisions you made yourself.

EXAMPLE AGENT DYNAMICS

A research agent identifies Crossref constraints; an implementation agent proposes a tool boundary; a review agent looks for unsupported-answer paths; a test agent suggests adversarial cases. You remain responsible for selecting and verifying the work.

Verification

Submit **VERIFICATION.md**. Automated tests must mock both the scholarly-data API and the model boundary; tests must not depend on live external services.

- A query returns correctly cited results from known API fixtures.
- A date or work-type constraint becomes the expected structured filter.
- Missing abstracts or author metadata are displayed safely.
- No results, API failures, rate limiting, and model timeouts are handled clearly.
- An attempt to make the system invent papers is rejected or safely ignored.
- A simulated model response that overclaims beyond source metadata is constrained, rejected, or surfaced as a test failure.
- A manual end-to-end check covers question, recommendation, trace inspection, save, and removal.

Submission

Submit a repository with the following structure. Equivalent organization is fine if the intent is clear.

```
README.md
DESIGN.md
AI_USAGE.md
VERIFICATION.md
<application source>
<tests>
```

Your README must include the live deployment URL, setup and run instructions, test commands, required environment variables, a short architecture overview, hosting notes, and known limitations.

The deployed application must be usable by reviewers without access to your private credentials. If the source API or model provider requires a key, configure it server-side and document any usage limits.

Evaluation rubric

Area	Weight	What strong looks like
Research and design	20%	Accurate API understanding, clear tradeoffs, explicit contracts and constraints.
Agent grounding and safety	25%	Evidence-driven output, transparent tool use, sensible uncertainty behavior.
Full-stack implementation	20%	Complete core flow, persistence, resilient error handling, clear UX.
Verification	20%	Useful fixtures, adversarial tests, clear manual checks, no reliance on live APIs.
AI-development explanation	10%	Deliberate prompts, well-defined roles, critical review and ownership.
Deployment	5%	Publicly reachable, usable application with clear configuration and hosting rationale.

Discussion prompts

- Why is your system an agent rather than a basic chat completion?
- How do tool output and evidence constrain the final response?
- Where can the system still fail, and how would you improve it with another day?
- Which development prompt was most useful, and which AI output required correction?

The project values clear reasoning, thoughtful engineering, and systems that can explain what they know - and what they do not.